

Vulnerability Assessment Report Web Application Vulnerability Testing

1. Introduction

Web applications are widely used today and often store sensitive user data. Because of this, they are common targets for cyber-attacks. The purpose of this vulnerability assessment was to understand how common web application vulnerabilities occur and how they can be identified using security testing tools.

This task focuses on practical testing of a deliberately vulnerable web application using **Burp Suite Community Edition**. The assessment mainly covers two critical vulnerabilities from the **OWASP Top 10** list: **SQL Injection** and **Cross-Site Scripting (XSS)**. By performing this assessment, hands-on experience was gained in intercepting web traffic, injecting malicious inputs, and analyzing application responses.

2. Scope of the Assessment

The testing was limited to a vulnerable web application hosted in a local environment. Only manual testing techniques were used, and no automated exploitation tools were involved.

Scope includes:

- Testing user input fields
 - Login and form functionalities
 - Detection of SQL Injection and XSS vulnerabilities **Out of scope:**
 - Denial of Service attacks
 - Server-side configuration testing
 - Network-level attacks
-

3. Tools and Environment

The following tools and environment were used during the assessment:

- **Burp Suite Community Edition** – Used to intercept, inspect, and modify HTTP requests and responses.

- **Web Browser (Chrome/Firefox)** – Used to interact with the application.
 - **Vulnerable Web Application** – DVWA / OWASP Juice Shop installed locally.
-

4. Testing Methodology

The vulnerability testing process followed a simple and structured approach:

1. Studied OWASP Top 10 vulnerabilities to understand common attack methods.
2. Set up the vulnerable web application in a local testing environment.
3. Configured the browser to route traffic through Burp Suite.
4. Intercepted HTTP requests using Burp Proxy.
5. Modified request parameters to inject malicious payloads.
6. Observed application responses to identify abnormal behavior.
7. Recorded the discovered vulnerabilities and their impact.
8. Suggested appropriate mitigation techniques.

This approach helped in understanding how attackers exploit insecure input handling in web applications.

5. Vulnerability Findings

5.1 SQL Injection

Description:

SQL Injection was identified in the login functionality of the application. The application did not properly validate user input before using it in SQL queries.

Payload Used:

' OR '1'='1

Result:

The application allowed login without valid credentials, indicating an authentication bypass.

Impact:

- Unauthorized access to the application
 - Exposure of sensitive database information
 - Possibility of data modification or deletion **Severity:** High
-

5.2 Cross-Site Scripting (XSS)

Description:

The application was vulnerable to Cross-Site Scripting due to improper input sanitization. User-supplied input was reflected back to the browser without encoding.

Payload Used:

```
<script>alert('XSS')</script>
```

Result:

The injected JavaScript executed successfully in the browser, confirming the presence of an XSS vulnerability.

Impact:

- Theft of user session cookies
- Execution of malicious scripts in the victim's browser
- Phishing and social engineering attacks

Severity: Medium

6. Risk Analysis

The identified vulnerabilities present serious security risks. SQL Injection can lead to complete compromise of the application's database, while XSS can be used to attack users directly. If these vulnerabilities exist in a real-world application, attackers could exploit them to steal data, manipulate information, or gain unauthorized access.

7. Recommended Mitigations

SQL Injection • Use

parameterized queries or prepared statements

- Validate and sanitize all user inputs
- Use least-privilege database user accounts
- Avoid displaying detailed database error messages

Cross-Site Scripting (XSS)

- Encode output before displaying it in the browser
- Sanitize user inputs
- Implement **Content Security Policy (CSP)**
- Avoid inline JavaScript where possible

8. Conclusion

This vulnerability assessment provided practical exposure to real-world web application security issues. Using Burp Suite, common vulnerabilities such as SQL Injection and XSS were successfully identified and analyzed. The task highlighted the importance of secure coding practices and regular security testing. Applying the recommended mitigations can significantly improve the security posture of web applications.

PREPARE BY

SAFFIQ MOHAMMED .S